

第 8 卷 竞赛数学参考

考试速查：主查 NOI/ICPC 风格数学补充：整除与取整、线性同余、exgcd、CRT、欧拉/莫比乌斯、容斥、矩阵、期望、SG 和几何精度。

Contents

第 8 卷 竞赛数学参考	1
考试速查定位	1
竞赛数学参考速查索引	2
MATHREF-00 竞赛数学参考总路由	2
数学题路由表	2
MATHREF-01 整除、同余与 gcd/lcm	3
1. 整除与同余	4
2. gcd/lcm 与 Bezout 直觉	5
MATHREF-02 扩展欧几里得、逆元与 CRT	6
1. 扩展欧几里得与线性同余方程	6
2. 模逆元	7
3. CRT 与扩展 CRT	9
MATHREF-03 筛法、质因数分解、欧拉函数与莫比乌斯	11
1. 素数筛与质因数分解	11
2. 欧拉函数 phi	12
3. 莫比乌斯函数与容斥基础	14
MATHREF-04 组合数学、容斥、抽屉与常见计数	15
1. 排列组合与二项式	15
2. Lucas 定理	16
3. 容斥	17
4. 鸽巢/抽屉原理	18
5. Catalan 与常见计数	19
MATHREF-05 递推、概率期望与博弈论	20
1. 递推与矩阵快速幂	20
2. 概率与期望基础	21
3. 博弈论 SG / 必胜必败	22
MATHREF-06 计算几何、数值误差与数学建模路由	24
1. 基础计算几何公式	24
2. 浮点误差与取整	25
3. 常见数学建模路由	26
MATH-LA-00 基础线性代数	27

第 8 卷 竞赛数学参考

考试速查定位

第 8 卷 竞赛数学参考

- **这是第几卷：** 08。
- **主要查什么：** 主查 NOI/ICPC 风格数学补充：整除与取整、线性同余、exgcd、CRT、欧拉/莫比乌斯、容斥、矩阵、期望、SG 和几何精度。
- **什么时候翻：** 数学题更深，涉及同余、CRT、莫比乌斯、矩阵、概率、博弈、几何时翻。
- **题面关键词：** 整除；同余；exgcd；CRT；欧拉/莫比乌斯；容斥；矩阵快速幂；期望；SG；几何。

竞赛数学参考速查索引

题面信号	模块入口
整除、同余、取模化简	MATHREF-01
扩展欧几里得、CRT、逆元	MATHREF-02
素数、因数、欧拉函数、莫比乌斯	MATHREF-03
组合计数、容斥、Lucas	MATHREF-04
递推、概率期望、博弈	MATHREF-05
几何、数值建模	MATHREF-06
矩阵/线性代数补充	MATH-LA-00

低优先级但可打印备用: FFT/NTT、BSGS、Miller-Rabin/Pollard-Rho、Burnside/Polya、Min_25 筛等不作为主模板, 遇到高阶真题时优先拿小数据/暴力/特判分。

MATHREF-00 竞赛数学参考总路由

模块编号: MATHREF-00

模块名称: 竞赛数学参考总路由

标签: [数学][NOI][ICPC][路由][纸质速查]

一句话用途: 把数学题的题面信号快速路由到第 6 卷和第 8 卷的可抄模块, 避免考场在数论、计数、概率、几何之间迷路。

题面触发词: 整除、余数、同余、互质、逆元、组合数、排列、容斥、质数、因子、欧拉函数、莫比乌斯、递推、矩阵、概率、期望、博弈、几何、精度。

什么时候用:

- 题目明显带数学公式、取模、计数或数值精度。
- 算法主干已经确定, 但需要补组合数、逆元、gcd、快速幂、筛法等工具。
- 不确定某个数学工具是否适用, 需要先查“不要什么时候用”和坑点。

不要什么时候用:

- 题目其实是图、DP、数据结构, 只是答案要取模; 先查主算法, 再接数学工具。
- 模数不是质数时, 不要直接套费马逆元。
- 数学工具看起来高级但不会改题意时, 先写暴力/特判/小数据。

复杂度:

- 见各子模块; 本路由只做选择, 不决定最终复杂度。
- 数学题也要估算“预处理复杂度 + 单次查询复杂度 + 数据范围”。

依赖的标准容器:

- `vector<int>` / `static int prime[MAXN]`: 筛法。
- `static ll fac[MAXN], ifac[MAXN]`: 组合数预处理。
- `pair<ll, ll>`: 扩展欧几里得返回系数。
- `long long` 和必要时 `__int128`: 乘法溢出防御。

接口:

```
gcd/lcm/quick_pow -> MATH-01/02
inverse/combinatoin/CRT -> MATH-03 或 MATHREF-02/04
prime/factor/phi/mobius -> MATH-04 或 MATHREF-03
recurrence/matrix/probability/game -> MATH-05 或 MATHREF-05
geometry/numeric -> MATHREF-06
linear algebra -> MATH-LA-00
```

模板代码: 本模块是路由表, 不放完整 C++ 模板; 具体可抄代码按路由结果进入对应 MATH-*, MATHREF-* 或 MATH-LA-* 模块。

数学题路由表

题面信号	优先模块	先问自己
最大公约数、互质、约分	MATH-01 / MATHREF-01	是否有负数？答案是否溢出？
$a^b \bmod p$ 、快速幂	MATH-02	乘法是否需要 <code>__int128</code> ？
取模除法、逆元	MATH-03 / MATHREF-02	模数是否为质数？分母是否和模数互质？
多个同余方程	MATHREF-02	模数是否两两互质？
多次 $C(n, k) \bmod p$	MATH-03 / MATHREF-04	n 最大值能否预处理？ p 是否质数？
容斥、至少一个、不能同时	MATHREF-04	集合数量是否很小？
判断质数、分解因数	MATH-04 / MATHREF-03	单次大数还是多次小数？
欧拉函数、莫比乌斯、整除分块	MATHREF-03	是否真的需要进阶数论，还是枚举因子可过？
线性递推第 n 项	MATH-05 / MATH-LA-00	n 是否很大，是否要矩阵快速幂？
概率、期望	MATHREF-05	状态是否有环，方程是否可直接列？
必胜/必败、取石子	MATHREF-05	是否是独立子游戏，是否要 SG？
点线距离、凸包、面积	MATHREF-06	精度要求和退化情况是什么？

常见坑：

- 看到“取模”不等于数学题；图/DP/数据结构题也会取模。
- $a / b \bmod p$ 不能写普通除法；要逆元，且逆元存在才行。
- $\text{lcm} = a / \gcd(a, b) * b$ 仍可能溢出；先除后乘，并用上限检查。
- 组合数里 $k < 0$ 或 $k > n$ 应返回 0。
- 质因数分解试除到 $p * p \leq x$ 时， $p * p$ 可能溢出，写 $p \leq x / p$ 。

暴力/部分分替代：

- 数论不会：小范围直接枚举因子、枚举答案或模拟。
- 组合不会：小 n 用 Pascal 三角或 DFS 枚举。
- 概率期望不会：小状态用记忆化搜索，或者模拟验证样例。
- 几何不会：先处理水平/垂直/共线/整数点等特判，拿部分分。

升级方向：

枚举 -> 公式/筛法
 单次计算 -> 预处理
 普通除法 -> 逆元/扩欧
 暴力计数 -> 容斥/DP/组合数
 线性递推模拟 -> 矩阵快速幂

最小测试样例：

`gcd`: $(0, 0)$, $(0, x)$, 负数
`comb`: $k=0$, $k=n$, $k>n$
`inverse`: $a=0$, $\gcd(a, \text{mod}) \neq 1$
`prime`: 1, 2, 平方数, 大质数
`geometry`: 共线、重合点、边界点

MATHREF-01 整除、同余与 gcd/lcm

本文件定位：竞赛数学知识查阅，不只是代码模板。看到题面后先判断“是否是整数关系/模关系/周期关系”。

调用示例：先按题面关键词定位到本文件的小节，再把对应 C++17 模板或计算方式 抄入主程序验证。

最小测试样例：每个公式先用 2 到 3 个手算小数值自测，例如模数为 5、余数为 0/1/4 的边界。

模块编号：MATHREF-01

模块名称：整除、同余与 gcd/lcm 参考

什么时候用：题目出现整除、余数、周期、取模、最大公约数、最小公倍数、互质等整数关系。

不要什么时候用：题目核心是组合数预处理、图论或 DP 时，本文件只提供数学判断和公式，不替代对应模板。

复杂度：公式查阅本身 $O(1)$ ；gcd 为 $O(\log \min(a, b))$ ；枚举因子通常到 $\sqrt{n}$ 。

依赖的标准容器：通常只需 `long long`；批量统计时可接 `vector<int>` 或筛法模块。

接口：`std::gcd(a, b)`、安全 lcm、同余判断、取模规范化。

1. 整除与同余

题面触发词：

- 整除、余数、取模、模 m 意义下相等。
- 周期、循环、每隔 k 次。
- 判断 a 和 b 除以 m 的余数是否相同。
- 输出答案对某个数取模。

使用条件：

- 所有量都是整数。
- 同余只关心余数，不关心原数大小。
- 模数 $m > 0$ 。

公式/结论：

$$a \equiv b \pmod{m} \iff m \mid (a-b)$$
$$(a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$
$$(a-b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$$
$$(a*b) \bmod m = ((a \bmod m) * (b \bmod m)) \bmod m$$

如果 $a \equiv b \pmod{m}$, $c \equiv d \pmod{m}$, 则：

$$a+c \equiv b+d \pmod{m}$$
$$a*c \equiv b*d \pmod{m}$$

C++17 模板或计算方式：

```
using ll = long long;
```

```
ll norm(ll x, ll mod) {  
    assert(mod > 0);  
    x %= mod;  
    if (x < 0) x += mod;  
    return x;  
}
```

```
ll floor_div(ll a, ll b) {  
    // b > 0, 向下取整  
    assert(b > 0);  
    __int128 aa = a, bb = b;  
    __int128 q = aa / bb;  
    __int128 r = aa % bb;  
    if (r != 0 && ((r > 0) != (bb > 0))) q--;  
    return (ll)q;  
}
```

```
ll ceil_div(ll a, ll b) {  
    // b > 0, 向上取整  
    assert(b > 0);  
    __int128 aa = a, bb = b;  
    __int128 q = aa / bb;  
    __int128 r = aa % bb;  
    if (r != 0 && ((r > 0) == (bb > 0))) q++;  
    return (ll)q;  
}
```

常见坑：

- C++ 里 $-3 \% 5 == -3$ ，不是 2；需要 `norm`。
- $1e9+7$ 不要写成浮点参与计算，写 `1000000007LL`。
- 整除要求余数为 0，不是浮点除法结果为整数。
- 地板除和 C++ / 对负数的截断不同。

暴力/部分替代：

- 小数据直接模拟每一步并记录余数。
- 不会推周期时，用 `vis[余数]` 找循环。

最小验错:

```
norm(-3, 5) = 2
7 ≡ 2 (mod 5)
floor_div(-3, 2) = -2
ceil_div(-3, 2) = -1
```

2. gcd/lcm 与 Bezout 直觉

题面触发词:

- 最大公约数、最小公倍数。
- 互质、约分、公共因子。
- 两个周期何时同时发生。
- 是否存在整数 x, y 使 $ax+by=c$ 。

使用条件:

- $\text{gcd}(a, b)$ 用于公共因子和互质判断。
- $\text{lcm}(a, b)$ 用于周期合并, 要求注意溢出。
- $ax+by=c$ 有整数解当且仅当 $\text{gcd}(a, b) \mid c$ 。

公式/结论:

```
gcd(a, b) = gcd(b, a mod b)
lcm(a, b) = |a / gcd(a, b) * b|
gcd(a, b) = 1 表示 a, b 互质。
```

Bezout:

存在整数 x, y , 使 $ax + by = \text{gcd}(a, b)$ 。

所以 $ax + by = c$ 有解 $\Leftrightarrow \text{gcd}(a, b) \mid c$ 。

C++17 模板或计算方式:

```
using ll = long long;

ll gcd_ll(ll a, ll b) {
    if (a == LLONG_MIN || b == LLONG_MIN) {
        unsigned long long x = a < 0 ? 0ULL - (unsigned long long)a : (unsigned long long)a;
        unsigned long long y = b < 0 ? 0ULL - (unsigned long long)b : (unsigned long long)b;
        while (y) {
            unsigned long long t = x % y;
            x = y;
            y = t;
        }
        return (ll)x;
    }
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a;
}

ll lcm_limit(ll a, ll b, ll limit) {
    if (a == 0 || b == 0) return 0;
    __int128 aa = a, bb = b;
    if (aa < 0) aa = -aa;
    if (bb < 0) bb = -bb;
    ll g = gcd_ll(a, b);
    aa /= g;
    __int128 lim = limit;
    ll over = (limit == LLONG_MAX ? limit : limit + 1);
    if (bb != 0 && aa > lim / bb) return over;
```

```

__int128 res = aa * bb;
if (res > lim) return over;
return (ll)res;
}

```

常见坑:

- lcm 先乘再除容易溢出。
- $\gcd(0, x) = |x|$ 。
- $\text{lcm}(0, x) = 0$ 。
- 周期合并很多次时，一旦超过题目上限就可以停止。

暴力/部分替代:

- 小数据从 $\min(a, b)$ 往下枚举 gcd。
- 小数据从 $\max(a, b)$ 往上枚举 lcm。
- 周期问题直接模拟到第一个同时出现的位置。

最小验错:

$\gcd(12, 18) = 6$

$\text{lcm}(12, 18) = 36$

$\gcd(0, 7) = 7$

方程 $6x + 10y = 8$ 有解, 因为 $\gcd = 2$ 且 $2 \mid 8$

方程 $6x + 10y = 9$ 无解

MATHREF-02 扩展欧几里得、逆元与 CRT

本文定位: 解决“模意义下除法”和“多个同余条件合并”。竞赛里看到除法先问模数是否为质数。

调用示例: 先判断模数是否质数; 质数模可用快速幂逆元, 非质数模优先用 exgcd/CRT 小节。

最小测试样例: $\text{exgcd}(30, 18)$ 、 $\text{inv_mod}(3, 11)$ 、 $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$ 都应能手算核对。

模块编号: MATHREF-02

模块名称: 扩展欧几里得、逆元与 CRT 参考

什么时候用: 题目要求模意义下除法、线性同余方程、 $ax + by = c$ 、多个余数条件合并。

不要什么时候用: 模数明确是质数且只是普通组合数, 可优先用 MATH-03 的费马逆元和阶乘逆元; 没有取模除法时不要硬套。

复杂度: 扩展欧几里得 $O(\log \min(a, b))$; CRT 合并每个方程一次 $O(k \log M)$ 。

依赖的标准容器: long long; 多方程 CRT 可用 `vector<pair<ll, ll>>` 存 (remainder, mod)。

接口: $\text{exgcd}(a, b, x, y)$ 、 $\text{mod_inverse}(a, \text{mod})$ 、线性同余求解、CRT 合并。

1. 扩展欧几里得与线性同余方程

题面触发词:

- 求整数解 $ax + by = c$ 。
- 求 $a \cdot x \equiv b \pmod m$ 。
- 模数不一定是质数。
- 同余方程、最小非负解。

使用条件:

- a, b, c, m 是整数。
- 解线性方程前先算 $g = \gcd(a, b)$ 或 $\gcd(a, m)$ 。
- $a \cdot x \equiv b \pmod m$ 有解当且仅当 $\gcd(a, m) \mid b$ 。

公式/结论:

$\text{exgcd}(a, b)$ 求 x, y , 使 $ax + by = \gcd(a, b)$ 。

$ax + by = c$:

设 $g = \gcd(a, b)$, 若 $c \% g \neq 0$, 无整数解。

否则 exgcd 得到 x_0, y_0 后, 乘 c/g 得一组解。

$a \cdot x \equiv b \pmod{m}$:
 等价于 $a \cdot x + m \cdot y = b$ 。
 $g = \gcd(a, m)$, 若 $b \% g \neq 0$, 无解。
 化成 $(a/g) \cdot x \equiv (b/g) \pmod{m/g}$ 。

C++17 模板或计算方式:

```

using ll = long long;

ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return a >= 0 ? a : -a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    __int128 yy = (__int128)x1 - (__int128)(a / b) * y1;
    assert((__int128)LLONG_MIN <= yy && yy <= (__int128)LLONG_MAX);
    y = (ll)yy;
    return g;
}

ll norm(ll x, ll mod) {
    assert(mod > 0);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

// 返回  $a \cdot x \equiv b \pmod{m}$  的一个最小非负解; 无解返回 -1
ll solve_linear_congruence(ll a, ll b, ll m) {
    assert(m > 0);
    ll x, y;
    ll g = exgcd(a, m, x, y);
    if (b % g != 0) return -1;
    ll mod2 = m / g;
    return norm((__int128)x * (b / g) % mod2, mod2);
}
  
```

常见坑:

- exgcd 返回的一组解不是唯一解。
- $a \cdot x \equiv b \pmod{m}$ 的解模数是 m/g 。
- a 或 b 为负时, exgcd 模板仍返回正 gcd, 最后用 norm 规范答案。
- 乘 $x \cdot (b/g)$ 可能溢出, 使用 `__int128`。

暴力/部分分替代:

- $m \leq 1e6$ 时枚举 $x=0..m-1$ 。
- 只判断是否有解时, 只需检查 $b \% \gcd(a, m) == 0$ 。

最小验错:

$14 \cdot x \equiv 30 \pmod{100}$
 $\gcd(14, 100) = 2, 30 \% 2 = 0$, 有解
 化简为 $7 \cdot x \equiv 15 \pmod{50}$
 一个解 $x=45$, 因为 $14 \cdot 45 = 630, 630 \% 100 = 30$

2. 模逆元

题面触发词:

- 模意义下除法。
- 求 $a^{-1} \pmod{m}$ 。
- 组合数预处理里的逆元。
- 概率分数取模。

使用条件:

- a 在模 m 下有逆元当且仅当 $\gcd(a, m) = 1$ 。
- 若 m 是质数且 $a \% m \neq 0$, 可用费马小定理。
- 若 m 不是质数, 用扩展欧几里得。

公式/结论:

$$a * \text{inv}(a) \equiv 1 \pmod{m}$$

m 是质数:

$$\text{inv}(a) = a^{(m-2)} \pmod{m}$$

m 不一定是质数:

用 exgcd 解 $ax + my = 1$, x 即逆元。

C++17 模板或计算方式:

```
using ll = long long;
```

```
ll exgcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return a >= 0 ? a : -a;
    }
    ll x1, y1;
    ll g = exgcd(b, a % b, x1, y1);
    x = y1;
    __int128 yy = (__int128)x1 - (__int128)(a / b) * y1;
    assert((__int128)LLONG_MIN <= yy && yy <= (__int128)LLONG_MAX);
    y = (ll)yy;
    return g;
}

ll norm(ll x, ll mod) {
    assert(mod > 0);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

ll mul_mod(ll a, ll b, ll mod) {
    assert(mod > 0);
    a %= mod;
    b %= mod;
    if (a < 0) a += mod;
    if (b < 0) b += mod;
    return (ll)((__int128)a * b % mod);
}

ll pow_mod(ll a, ll e, ll mod) {
    assert(mod > 0);
    assert(e >= 0);
    ll res = 1 % mod;
    a %= mod;
    if (a < 0) a += mod;
    while (e > 0) {
        if (e & 1) res = mul_mod(res, a, mod);
        a = mul_mod(a, a, mod);
        e >>= 1;
    }
    return res;
}
```



```

11 inv_prime(11 a, 11 mod) {
    assert(mod > 1);
    a = norm(a, mod);
    if (a == 0) return -1; // 不存在逆元
    return pow_mod(a, mod - 2, mod);
}

11 inv_exgcd(11 a, 11 mod) {
    assert(mod > 0);
    11 x, y;
    11 g = exgcd(a, mod, x, y);
    if (g != 1) return -1;
    return norm(x, mod);
}

```

常见坑:

- mod 不保证质数时不能用 $a^{(mod-2)}$ 。
- $a \% mod == 0$ 没有逆元; 本模板的 `inv_prime` 会返回 -1。
- 分数取模不是普通整数除法。
- 组合数阶乘逆元在非质数模数下可能整体失效。

暴力/部分替代:

- $mod \leq 1e6$ 时枚举 x 找 $a*x \% mod == 1$ 。
- 小范围组合数用 Pascal 递推, 不用逆元。

最小验错:

$mod=7, inv(3)=5$, 因为 $3*5\%7=1$
 $mod=8, inv(3)=3$, 因为 $3*3\%8=1$
 $mod=8, inv(2)$ 不存在, 因为 $gcd(2,8)=2$

3. CRT 与扩展 CRT

题面触发词:

- 同时满足多个余数条件。
- $x \equiv a_i \pmod{m_i}$ 。
- 多个周期同时对齐。
- 模数两两互质/不一定互质。

使用条件:

- CRT 标准版要求模数两两互质。
- 扩展 CRT 不要求互质, 但每次合并要检查是否兼容。
- 结果通常输出最小非负解。

公式/结论:

两个方程:

$x \equiv r_1 \pmod{m_1}$
 $x \equiv r_2 \pmod{m_2}$

令 $x = r_1 + m_1*k$

代入第二个:

$m_1*k \equiv r_2 - r_1 \pmod{m_2}$

有解条件:

$gcd(m_1, m_2) \mid (r_2 - r_1)$

新模数:

$lcm(m_1, m_2) = m_1/g*m_2$

C++17 模板或计算方式:

```

using 11 = long long;

11 exgcd(11 a, 11 b, 11 &x, 11 &y) {
    if (b == 0) {

```

```

    x = (a >= 0 ? 1 : -1);
    y = 0;
    return a >= 0 ? a : -a;
}

ll x1, y1;
ll g = exgcd(b, a % b, x1, y1);
x = y1;
__int128 yy = (__int128)x1 - (__int128)(a / b) * y1;
assert((__int128)LLONG_MIN <= yy && yy <= (__int128)LLONG_MAX);
y = (ll)yy;
return g;
}

ll norm(ll x, ll mod) {
    assert(mod > 0);
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

bool merge_crt(ll &r1, ll &m1, ll r2, ll m2) {
    // 当前  $x \equiv r1 \pmod{m1}$ , 合并  $x \equiv r2 \pmod{m2}$ 
    assert(m1 > 0 && m2 > 0);
    ll x, y;
    ll g = exgcd(m1, m2, x, y);
    __int128 diff = (__int128)r2 - r1;
    if (diff % g != 0) return false;

    ll mod2 = m2 / g;
    __int128 left = (diff / g) % mod2;
    if (left < 0) left += mod2;
    ll k = (ll)(left * norm(x, mod2) % mod2);

    __int128 nr = (__int128)r1 + (__int128)m1 * k;
    __int128 nm = (__int128)m1 / g * m2;
    if (nm <= 0 || nm > LLONG_MAX) return false; // 本模板只返回 Long Long 范围内的合并模数
    r1 = (ll)(nr % nm);
    if (r1 < 0) r1 += (ll)nm;
    m1 = (ll)nm;
    return true;
}

// 方程编号 1..k:  $x \equiv r[i] \pmod{m[i]}$ 
// 返回 {是否有解, 最小非负解}
pair<bool, ll> excrt(ll r[], ll m[], int k) {
    if (k <= 0) return {false, -1};
    for (int i = 1; i <= k; i++) {
        if (m[i] <= 0) return {false, -1};
    }
    ll ans = r[1], mod = m[1];
    ans = norm(ans, mod);
    for (int i = 2; i <= k; i++) {
        if (!merge_crt(ans, mod, norm(r[i], m[i]), m[i])) {
            return {false, -1};
        }
    }
    return {true, ans};
}

```

常见坑:

- 模数不互质时, 普通 CRT 公式可能错, 必须用扩展 CRT。
- 兼容条件是 $(r2-r1) \% \gcd(m1,m2) == 0$ 。
- 新模数是 lcm, 可能溢出。

- 余数先规范到 $[0, m-1]$ 。

暴力/部分分替代:

- 模数乘积小: 从 $x=r1$ 开始每次加 $m1$ 枚举。
- 条件少且上界小: 直接枚举 $x=0..limit$ 检查所有同余。

最小验错:

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

最小非负解 $x=8$

$$x \equiv 1 \pmod{2}$$

$$x \equiv 0 \pmod{4}$$

无解, 因为 $\gcd(2,4)=2$ 不能整除 -1

MATHREF-03 筛法、质因数分解、欧拉函数与莫比乌斯

本文件定位: 数论预处理和乘法函数查阅。优先掌握筛法、分解、phi; 莫比乌斯作为中高阶补充。

调用示例: $n \leq 1e7$ 先考虑筛; 单个大数分解用试除或 SPF; 互质计数再接 phi。

最小测试样例: $12 = 2^2 * 3$, $\phi(12)=4$, $\mu(1)=1$, $\mu(4)=0$, $\mu(6)=1$ 。

模块编号: MATHREF-03

模块名称: 筛法、质因数分解、欧拉函数与莫比乌斯参考

什么时候用: 题目出现质数、分解质因数、约数、互质计数、欧拉函数、莫比乌斯或多次数论预处理。

不要什么时候用: 只需要一次 $\gcd/lcm$ 或快速幂时, 优先查 MATHREF-01 或 MATH-02。

复杂度: 埃氏筛约 $O(n \log \log n)$; 线性筛 $O(n)$; 单个数试除 $O(\sqrt{n})$; 用最小质因子分解约 $O(\log n)$ 。

依赖的标准容器: `vector<int>`、`vector<bool>` 或 `vector<char>`; 分解结果常用 `vector<pair<ll,int>>`。

接口: 筛质数、最小质因子、质因数分解、 $\phi(n)$ 、莫比乌斯函数参考。

1. 素数筛与质因数分解

题面触发词:

- 质数、素数、合数。
- 分解质因数、约数个数、约数和。
- 多次询问某数是否是质数。
- 统计 $1..n$ 中的质数。

使用条件:

- N 不太大时可筛, 常见 $N \leq 1e7$ 。
- 多次分解 $x \leq N$ 时用最小质因子 `spf`。
- 单个大数可用质数表试除到 $\sqrt{x}$ 。

公式/结论:

$$n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$$

$$\text{约数个数 } d(n) = (a_1+1)(a_2+1)\dots(a_k+1)$$

$$\text{约数和 } \sigma(n) = (1+p_1+\dots+p_1^{a_1}) * \dots$$

C++17 模板或计算方式:

```
using ll = long long;
```

```
struct Sieve {
    int N = 0;
    vector<int> primes, spf;
```

```
    void init(int n) {
        N = n;
        primes.clear();
```

```

    spf.assign(N + 1, 0);
    for (int i = 2; i <= N; i++) {
        if (!spf[i]) {
            spf[i] = i;
            primes.push_back(i);
        }
        for (int p : primes) {
            if (p > spf[i] || (ll)i * p > N) break;
            spf[i * p] = p;
        }
    }
}

bool is_prime(int x) const {
    return x >= 2 && x <= N && spf[x] == x;
}

vector<pair<int,int>> factor_spf(int x) const {
    assert(2 <= x && x <= N);
    vector<pair<int,int>> res;
    while (x > 1) {
        int p = spf[x], c = 0;
        while (x % p == 0) {
            x /= p;
            c++;
        }
        res.push_back({p, c});
    }
    return res;
}
};

```

常见坑:

- 1 不是质数。
- spf 只能分解 $\leq N$ 的数。
- 试除判断 $p * p \leq x$ 时要转 long long。
- N 太大不要强行开数组。

暴力/部分替代:

- 单次判断质数: 枚举 $2 \dots \sqrt{n}$ 。
- 单次分解: 从 2 开始试除。
- 小范围统计质数: 双重循环标记合数。

最小验错:

20 以内质数: 2 3 5 7 11 13 17 19

$360 = 2^3 * 3^2 * 5$

约数个数 = $4 * 3 * 2 = 24$

2. 欧拉函数 phi

题面触发词:

- $1 \dots n$ 中与 n 互质的数有多少个。
- 欧拉函数、互质计数。
- 费马/欧拉降幂。
- 分数环、循环节、原根相关基础题。

使用条件:

- $\phi(n)$ 表示 $1 \dots n$ 中与 n 互质的正整数个数。
- 需要质因数分解或筛法。
- 欧拉定理要求 $\gcd(a, m) = 1$ 。

公式/结论:

若 $n = p_1^{a_1} * p_2^{a_2} * \dots * p_k^{a_k}$
 $\phi(n) = n * (1-1/p_1) * (1-1/p_2) * \dots * (1-1/p_k)$

若 $\gcd(a,m)=1$:
 $a^{\phi(m)} \equiv 1 \pmod{m}$

质数 p :
 $\phi(p) = p-1$

C++17 模板或计算方式:

```
using ll = long long;

ll phi_one(ll n) {
    ll ans = n;
    for (ll p = 2; p <= n / p; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            ans = ans / p * (p - 1);
        }
    }
    if (n > 1) ans = ans / n * (n - 1);
    return ans;
}

vector<int> phi_sieve(int n) {
    vector<int> phi(n + 1), primes, is_comp(n + 1, 0);
    if (n >= 1) phi[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_comp[i]) {
            primes.push_back(i);
            phi[i] = i - 1;
        }
        for (int p : primes) {
            if ((ll)i * p > n) break;
            is_comp[i * p] = 1;
            if (i % p == 0) {
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
    return phi;
}
```

常见坑:

- 欧拉定理必须有 $\gcd(a,m)=1$ 。
- $\phi[1]=1$ 是常用约定。
- $\text{ans} = \text{ans} / p * (p-1)$ 顺序不要写反, 避免中间非整数和溢出。
- 大数 $p * p \leq n$ 要注意溢出。

暴力/部分替代:

- 小数据直接枚举 $i=1..n$, 统计 $\gcd(i,n)=1$ 。
- 多次小范围询问可预处理 $\gcd$ 表或直接筛。

最小验错:

$\phi(1)=1$
 $\phi(5)=4$
 $\phi(12)=4$, 对应 1,5,7,11

3. 莫比乌斯函数与容斥基础

题面触发词：

- 互质对数、 $\gcd(i, j)=1$ 的计数。
- 莫比乌斯、Mobius、反演。
- 容斥、没有被任何质因子整除。
- 平方因子、无平方因子数。

使用条件：

- 入门只需记住 $\mu(n)$ 与质因数关系。
- 真正的莫比乌斯反演属于中高阶；考试时间紧时优先容斥/暴力拿分。
- 常见用于把 $\gcd=1$ 转成按公因子求和。

公式/结论：

$\mu(1)=1$

若 n 含有平方质因子，则 $\mu(n)=0$

若 n 是 k 个不同质数的乘积，则 $\mu(n)=(-1)^k$

基础结论：

$\sum_{d|n} \mu(d) = 1, n=1$

$\sum_{d|n} \mu(d) = 0, n>1$

互质对数：

$\text{count}(\gcd(i, j)=1, 1 \leq i \leq n, 1 \leq j \leq m)$

$= \sum_{d=1.. \min(n, m)} \mu(d) * \text{floor}(n/d) * \text{floor}(m/d)$

C++17 模板或计算方式：

```
vector<int> mobius_sieve(int n) {
    vector<int> mu(n + 1), primes, is_comp(n + 1, 0);
    if (n >= 1) mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!is_comp[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if ((long long)i * p > n) break;
            is_comp[i * p] = 1;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
    return mu;
}
```

```
long long count_coprime_pairs(int n, int m) {
    if (n <= 0 || m <= 0) return 0;
    int lim = min(n, m);
    vector<int> mu = mobius_sieve(lim);
    __int128 ans = 0;
    for (int d = 1; d <= lim; d++) {
        ans += (__int128)mu[d] * (n / d) * (m / d);
    }
    return (long long)ans;
}
```

常见坑：

- μ 不是欧拉函数。
- 含平方因子时 $\mu=0$ 。

- 互质对公式里的 $\text{floor}(n/d)$ 是整数除法。
- 反演公式容易套错，基础卷只建议用于熟悉模型。

暴力/部分替代：

- $n, m \leq 3000$ ：双重循环统计 $\text{gcd}(i, j) == 1$ 。
- 条件少时直接用容斥枚举质因子集合。

最小验错：

```
mu(1)=1
mu(2)=-1
mu(6)=1
mu(12)=0, 因为含  $2^2$ 
1..2 与 1..2 的互质有 (1,1)(1,2)(2,1), 共 3
```

MATHREF-04 组合数学、容斥、抽屉与常见计数

本文定位：纸质查阅常用计数结论。先判断“有序/无序、可重复/不可重复、是否有禁区、是否取模”。

调用示例：先判断是否“选 k 个”“排列”“不相邻”“括号/合法序列”，再接组合数、容斥或 Catalan 小节。

最小测试样例： $C(5, 2)=10$, $P(5, 2)=20$, $\text{Catalan}(3)=5$ 。

模块编号：MATHREF-04

模块名称：组合数学、容斥、抽屉与常见计数参考

什么时候用：题目问排列组合、路径条数、二项式系数、容斥、抽屉原理、Catalan 或常见计数模型。

不要什么时候用：题目有明显顺序状态依赖时，优先看 DP-20 计数 DP；只求普通 $C(n, k)$ 代码时看 MATH-03。

复杂度：公式型 $O(1)$ ；Pascal 预处理 $O(n^2)$ ；阶乘逆元预处理 $O(n)$ ；容斥通常 $O(2^m)$ 。

依赖的标准容器：`vector<ll>`、`vector<vector<ll>>`；小集合容斥可用 `bitmask`。

接口：排列组合公式、Pascal、Lucas 入口、容斥、抽屉、Catalan 常见结论。

1. 排列组合与二项式

题面触发词：

- 从 n 个中选 k 个。
- 排列、组合、有序、无序。
- 二项式系数、 $C(n, k)$ 。
- 路径条数，只能向右/向下走。

使用条件：

- 组合：选出集合，不关心顺序。
- 排列：选出并排列，关心顺序。
- 阶乘逆元要求模数是质数；非质数先考虑 Pascal。

公式/结论：

```
P(n, k) = n! / (n - k)!
C(n, k) = n! / (k! (n - k)!)
C(n, k) = C(n, n - k)
C(n, k) = C(n - 1, k - 1) + C(n - 1, k)
```

二项式：

$$(x+y)^n = \sum C(n, k) x^k y^{n-k}$$

网格从 $(1, 1)$ 到 (n, m) ，只向下/右：

路径数 = $C(n+m-2, n-1)$

C++17 模板或计算方式：

```
using ll = long long;
```

```
vector<vector<ll>> comb_pascal(int N, ll mod) {
```

```

vector<vector<ll>> C(N + 1, vector<ll>(N + 1, 0));
for (int i = 0; i <= N; i++) {
    C[i][0] = C[i][i] = 1 % mod;
    for (int j = 1; j < i; j++) {
        C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % mod;
    }
}
return C;
}

```

常见坑:

- $C(n, k)$ 中 $k < 0$ 或 $k > n$ 是 0。
- 排列和组合不要混。
- 取模除法不能直接用 $/$ 。
- n 大时 Pascal 是 $O(n^2)$, 只能小范围。

暴力/部分替代:

- $n \leq 20$: DFS 枚举选/不选。
- $N \leq 2000$: Pascal 递推。
- 查询少且 k 小: 用整数约分乘法计算。

最小验错:

$C(5, 2) = 10$
 $P(5, 2) = 20$
 $C(5, 0) = 1$
 3×3 网格路径数 = $C(4, 2) = 6$

2. Lucas 定理

题面触发词:

- n, k 极大, 模数是小质数。
- 求 $C(n, k) \bmod p$, p 是质数。
- 多次大组合数查询。

使用条件:

- 模数 p 必须是质数。
- 通常 p 较小, 可以预处理 $0..p-1$ 的组合数。
- n, k 可很大, 用 p 进制拆位。

公式/结论:

把 n, k 写成 p 进制:

$n = n_0 + n_1 \cdot p + n_2 \cdot p^2 + \dots$
 $k = k_0 + k_1 \cdot p + k_2 \cdot p^2 + \dots$

Lucas:

$C(n, k) \bmod p = \text{product } C(n_i, k_i) \bmod p$

若某位 $k_i > n_i$, 则答案为 0。

C++17 模板或计算方式:

```

using ll = long long;

ll mod_pow(ll a, ll e, ll mod) {
    ll r = 1 % mod;
    while (e) {
        if (e & 1) r = r * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return r;
}

struct Lucas {
    ll p;

```



```

vector<ll> fac, ifac;

void init(ll prime_mod) {
    p = prime_mod;
    assert(p <= INT_MAX);
    fac.assign(p, 1);
    ifac.assign(p, 1);
    for (int i = 1; i < p; i++) fac[i] = fac[i - 1] * i % p;
    ifac[p - 1] = mod_pow(fac[p - 1], p - 2, p);
    for (int i = (int)p - 1; i >= 1; i--) ifac[i - 1] = ifac[i] * i % p;
}

ll smallC(ll n, ll k) const {
    if (k < 0 || k > n) return 0;
    return fac[n] * ifac[k] % p * ifac[n - k] % p;
}

ll C(ll n, ll k) const {
    if (k < 0 || k > n) return 0;
    ll ans = 1;
    while (n > 0 || k > 0) {
        ll ni = n % p, ki = k % p;
        if (ki > ni) return 0;
        ans = ans * smallC(ni, ki) % p;
        n /= p;
        k /= p;
    }
    return ans;
}
};

```

常见坑:

- Lucas 只适用于质数模数。
- p 太大时不能开 fac[p]。
- 每一位如果 $k_i > n_i$, 直接 0。
- fac 下标是 $0..p-1$, p 很大要转 int 风险。

暴力/部分分替代:

- $n \leq 2000$: Pascal。
- 查询少: 用乘法公式加约分。
- 模数不是质数: Lucas 不适用, 先拿小数据部分分。

最小验错:

$C(5, 2) \bmod 3 = 10 \bmod 3 = 1$
 $5 = (12)_3, 2 = (02)_3$
 $C(2, 2) * C(1, 0) = 1$

3. 容斥

题面触发词:

- 至少一个、没有任何一个。
- 不包含某些元素。
- 能被多个数中至少一个整除。
- 计算并集大小。

使用条件:

- 条件数量不大, 或交集大小容易算。
- 能计算每个条件子集同时满足的数量。
- $m \leq 20$ 是常见子集容斥范围。

公式/结论:

$|A_1 \cup A_2 \cup \dots \cup A_m|$
 $= \sum |A_i|$

```

- sum |Ai intersect Aj|
+ sum |Ai intersect Aj intersect Ak|
- ...

```

奇数个集合交集加，偶数个集合交集减。

C++17 模板或计算方式：

```

long long inclusion_exclusion_basic(int m, function<long long(long long)> count_intersection) {
    assert(0 <= m && m <= 60);
    __int128 ans = 0;
    for (long long mask = 1; mask < (1LL << m); mask++) {
        int bits = __builtin_popcountll((unsigned long long)mask);
        long long cur = count_intersection(mask);
        if (bits & 1) ans += cur;
        else ans -= cur;
    }
    return (long long)ans;
}

```

常见坑：

- 符号反了。
- 空集不参与“至少一个”的容斥。
- 交集计数写错比容斥公式本身更常见。
- $1 < m$ 要求 m 不大。

暴力/部分分替代：

- 枚举每个对象，逐个检查是否满足条件。
- 用布尔数组标记被覆盖对象。

最小验错：

1..10 中能被 2 或 3 整除：

$\text{floor}(10/2) + \text{floor}(10/3) - \text{floor}(10/6) = 5 + 3 - 1 = 7$

4. 鸽巢/抽屉原理

题面触发词：

- 至少有两个相同。
- 证明一定存在。
- $n+1$ 个物品放进 n 个盒子。
- 余数、颜色、生日、前缀和模 m 。

使用条件：

- 要证明存在性，而不是构造复杂对象。
- 能把对象映射到有限个“盒子”。
- 对前缀和取模是常见盒子。

公式/结论：

普通抽屉：

$n+1$ 个物品放入 n 个盒子，至少一个盒子有 ≥ 2 个物品。

加强版：

N 个物品放入 k 个盒子，至少一个盒子有 $\text{ceil}(N/k)$ 个物品。

前缀和结论：

若有 n 个数，考虑前缀和 mod n 。

若某个前缀余数为 0，存在和能被 n 整除的前缀。

否则 n 个非零余数落入 $n-1$ 个盒子，必有两个相同，相减得到一段和能被 n 整除。

C++17 模板或计算方式：

```

// 找一段非空子数组，其和能被 n 整除。返回 1-index 闭区间。
pair<int,int> subarray_sum_divisible_by_n(const vector<long long> &a) {
    int n = (int)a.size() - 1;
    if (n <= 0) return {-1, -1};

```

```

vector<int> first(n, -1);
long long sum = 0;
first[0] = 0;
for (int i = 1; i <= n; i++) {
    sum = (sum + a[i] % n) % n;
    if (sum < 0) sum += n;
    if (first[sum] != -1) return {first[sum] + 1, i};
    first[sum] = i;
}
return {-1, -1};
}

```

常见坑:

- 抽屉原理只保证存在，不一定直接给构造。
- 盒子数量要数对。
- 前缀和余数有 $0 \dots n-1$ 共 n 种。
- 题目要求输出方案时，要保存第一次出现位置。

暴力/部分替代:

- 枚举所有区间和，检查是否满足。
- 小数据直接搜索构造。

最小验错:

1-index 数组 $a=\{0,1,2,3\}$, $n=3$

前缀和 mod 3: 1,0

[1,2] 的和 3 能被 3 整除

5. Catalan 与常见计数

题面触发词:

- 合法括号序列。
- 出栈序列、二叉树形态。
- 不越过对角线的路径。
- 将凸多边形三角剖分。

使用条件:

- 典型 Catalan 结构: 每个前缀都不“欠账”。
- 常见规模不大可 DP, 大规模需要组合数。
- 取模组合公式要求模数处理正确。

公式/结论:

$Catalan(n) = 1/(n+1) * C(2n,n)$

$Catalan(n) = C(2n,n) - C(2n,n+1)$

递推:

$cat[0]=1$

$cat[n] = \sum_{i=0 \dots n-1} cat[i] * cat[n-1-i]$

前几项:

1, 1, 2, 5, 14, 42

C++17 模板或计算方式:

```

vector<long long> catalan_dp(int N, long long mod) {
    assert(mod > 0);
    vector<long long> cat(N + 1, 0);
    cat[0] = 1 % mod;
    for (int n = 1; n <= N; n++) {
        for (int i = 0; i < n; i++) {
            cat[n] = (cat[n] + (__int128)cat[i] * cat[n - 1 - i]) % mod;
        }
    }
    return cat;
}

```

常见坑:

- $1/(n+1)$ 是模意义除法, 不能直接整数除再取模, 除非先算整数大数。
- 合法括号要求任意前缀左括号数不少于右括号数。
- Catalan 模型很多, 但别把所有路径题都套 Catalan。

暴力/部分分替代:

- $n \leq 10$: 枚举所有括号序列检查合法。
- $N \leq 5000$: Catalan DP。
- 大规模且不会逆元: 用 $C(2n, n) - C(2n, n+1)$ 配合 Pascal 小范围。

最小验错:

```
n=0 -> 1
n=1 -> 1
n=2 -> 2
n=3 -> 5
```

MATHREF-05 递推、概率期望与博弈论

本文件定位: 把“状态随步骤变化”“随机过程”“必胜必败”三类数学建模快速分流。

调用示例: 线性递推先试矩阵快速幂; 概率题先定义 $dp[state]$ 期望; 博弈题先找必胜/必败状态。

最小测试样例: 斐波那契 $F(1)=1, F(2)=1, F(5)=5$; 只有一步可走的博弈先手必胜。

模块编号: MATHREF-05

模块名称: 递推、概率期望与博弈论参考

什么时候用: 题目出现第 n 项、固定递推、随机过程、期望值、概率、先手必胜/必败。

不要什么时候用: 普通最短路、背包或区间 DP 不需要概率/博弈公式; 递推带 $\max/\min$ 时不是矩阵快速幂。

复杂度: 普通递推 $O(n \cdot \text{状态数})$; 矩阵快速幂 $O(k^3 \log n)$; 小状态博弈 DP 为状态数乘转移数。

依赖的标准容器: `vector<double>`、`vector<ll>`、矩阵可用 `vector<vector<ll>>`。

接口: 线性递推、矩阵快速幂入口、期望 DP 基础方程、Nim/SG 入门判断。

1. 递推与矩阵快速幂

题面触发词:

- 第 n 项、第 n 天、第 n 次操作。
- 斐波那契、线性递推。
- n 很大, 例如 $1e18$ 。
- 固定转移重复执行。

使用条件:

- 普通递推: n 不大, 可以从小到大推。
- 矩阵快速幂: 状态转移是固定线性关系。
- 转移只含加法和乘法, 不含 $\max/\min$ 。

公式/结论:

线性递推例:

$$f[n] = c_1 \cdot f[n-1] + c_2 \cdot f[n-2] + \dots + c_k \cdot f[n-k]$$

写成矩阵:

$$\text{state}[n] = T \cdot \text{state}[n-1]$$

$$\text{state}[n] = T^{(n-\text{base})} \cdot \text{state}[\text{base}]$$

Fibonacci:

$$[F_n, F_{n-1}]^T = [[1, 1], [1, 0]]^{(n-1)} \cdot [F_1, F_0]^T$$

C++17 模板或计算方式:

```

using ll = long long;

struct Mat {
    int n;
    ll mod;
    vector<vector<ll>> a;
    Mat(int n_, ll mod_) : n(n_), mod(mod_), a(n_, vector<ll>(n_, 0)) {}
};

Mat mul(const Mat &A, const Mat &B) {
    Mat C(A.n, A.mod);
    for (int i = 0; i < A.n; i++) {
        for (int k = 0; k < A.n; k++) if (A.a[i][k]) {
            for (int j = 0; j < A.n; j++) {
                C.a[i][j] = (C.a[i][j] + (__int128)A.a[i][k] * B.a[k][j]) % A.mod;
                if (C.a[i][j] < 0) C.a[i][j] += A.mod;
            }
        }
    }
    return C;
}

Mat mat_pow(Mat A, long long e) {
    Mat R(A.n, A.mod);
    for (int i = 0; i < A.n; i++) R.a[i][i] = 1 % A.mod;
    while (e) {
        if (e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}

```

常见坑:

- 初始状态向量顺序必须与矩阵行列一致。
- $n=0/1$ 边界先单独处理。
- 矩阵模板常用 0-index。
- 非线性转移不能套矩阵乘法。

暴力/部分分替代:

- $n \leq 1e7$: 直接递推。
- 维度太大: 保留普通 DP 或找矩阵稀疏优化。

最小验错:

$F_0=0, F_1=1$

$F_2=1, F_3=2, F_{10}=55$

2. 概率与期望基础

题面触发词:

- 概率、随机、等概率。
- 期望、平均次数、期望得分。
- 抽卡、掷骰子、随机游走。
- 成功概率、失败概率。

使用条件:

- 概率总和为 1。
- 期望可以用线性性: 总期望等于各部分期望之和, 不要求独立。
- 若状态会转移, 常写期望 DP 或方程。

公式/结论:

概率:

$P(A^c)=1-P(A)$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$\text{独立时 } P(A \cap B) = P(A)P(B)$$

期望:

$$E[X] = \sum x * P(X=x)$$

$$\text{线性性: } E[X+Y] = E[X] + E[Y]$$

几何分布:

每次成功概率 p , 直到第一次成功的期望次数 = $1/p$

C++17 模板或计算方式:

```
// 骰子走到 >=n 的期望步数示意: dp[i] 表示从 i 到终点的期望
vector<double> dice_expectation(int n) {
    vector<double> dp(n + 6, 0.0);
    for (int i = n - 1; i >= 0; i--) {
        dp[i] = 1.0;
        for (int d = 1; d <= 6; d++) dp[i] += dp[i + d] / 6.0;
    }
    return dp;
}
```

有自环时的移项模板:

$$E[x] = 1 + p_{\text{self}} * E[x] + \sum(p_{\text{to}} * E[\text{to}])$$

$$\Rightarrow E[x] = (1 + \sum(p_{\text{to}} * E[\text{to}])) / (1 - p_{\text{self}})$$

```
double expectation_with_self_loop(double p_self, const vector<pair<double, double>>& next_exp) {
    const double EPS = 1e-12;
    if (p_self < -EPS || p_self > 1.0 + EPS) throw runtime_error("bad probability");
    if (1.0 - p_self < EPS) return 1e100; // 按题意可能是无穷大或不可达
    double rhs = 1.0;
    for (auto [p, e] : next_exp) {
        if (p < -EPS || p > 1.0 + EPS) throw runtime_error("bad probability");
        rhs += p * e;
    }
    return rhs / (1.0 - p_self);
}
```

考场口令: 如果转移会“留在原状态”, 不要直接递推; 先把 $E[x]$ 项移到等号左边。

常见坑:

- 期望线性性不要求独立, 但概率乘法通常要求独立。
- 有自环的期望 DP 可能要移项解方程。
- 输出小数注意精度; 输出模数下概率时要用逆元。
- double 比较不要直接 ==。

暴力/部分分替代:

- 小状态随机过程可模拟很多次估计, 但只能拿部分分/调试。
- 小数据枚举所有随机路径。
- 期望方程不会解时, 先写有限步 DP 截断。

最小验错:

公平硬币正面概率 $1/2$

直到第一次正面的期望次数 = 2

掷一次骰子的期望点数 = $(1+2+3+4+5+6)/6 = 3.5$

3. 博弈论 SG / 必胜必败

题面触发词:

- 两人轮流操作。
- 不能操作者输。
- 必胜、必败、先手、后手。
- 取石子、Nim、SG 函数。
- 多个独立游戏合并。

使用条件:

- 标准公平组合游戏: 两人可选操作相同, 无随机, 无隐藏信息, 不能操作者输。
- 状态图有向无环或可按大小递推。
- 多个独立子游戏用 SG 异或。

公式/结论:

必败态: 没有后继, 或所有后继都是必胜态。

必胜态: 存在一个后继是必败态。

$SG(x) = \text{mex}\{SG(y) \mid y \text{ 是 } x \text{ 的后继}\}$

多个独立游戏:

$SG_total = SG1 \text{ xor } SG2 \text{ xor } \dots$

$SG_total = 0 \rightarrow$ 先手必败

$SG_total \neq 0 \rightarrow$ 先手必胜

Nim:

若所有石子堆大小 xor 为 0, 先手必败; 否则先手必胜。

C++17 模板或计算方式:

```
int mex_value(const vector<int> &vals) {
    vector<int> seen(vals.size() + 2, 0);
    for (int x : vals) if (0 <= x && x < (int)seen.size()) seen[x] = 1;
    for (int i = 0; ; i++) if (!seen[i]) return i;
}

vector<int> sg_take_stones(int N, const vector<int> &moves) {
    vector<int> sg(N + 1, 0);
    for (int x = 1; x <= N; x++) {
        vector<int> nxt;
        for (int mv : moves) {
            if (mv <= 0) continue;
            if (x >= mv) nxt.push_back(sg[x - mv]);
        }
        sg[x] = mex_value(nxt);
    }
    return sg;
}
```

常见坑:

- SG 只适用于公平组合游戏, 不适合有随机/信息不对称/双方操作不同的题。
- “不能操作者赢”的 misere 规则不同。
- 多堆合并是异或, 不是加法。
- 状态有环时不能直接递推 SG。

暴力/部分替代:

- 小状态 DFS 判断 win/lose。
- 小堆数枚举所有操作搜索。
- Nim 题不会 SG 时先算所有堆 xor。

最小验错:

取石子, 每次取 1 或 2, 0 为必败

$sg[0]=0$

$sg[1]=1$

$sg[2]=2$

$sg[3]=0$

Nim piles=[1,2,3], xor=0, 先手必败

MATHREF-06 计算几何、数值误差与数学建模路由

本文件定位：整理基础公式和路由，不追求完整计算几何库。考试中几何题先画图，再确认整数/浮点/精度要求。

调用示例：点/向量题先抄 Point 运算；直线方向用叉积；整数除法边界用安全 floor_div/ceil_div。

最小测试样例：cross((1,0),(0,1))=1, floor_div(-3,2)=-2, ceil_div(-3,2)=-1。

模块编号：MATHREF-06

模块名称：计算几何、数值误差与数学建模路由参考

什么时候用：题目出现点线面、距离、面积、方向、浮点误差、取整、数学建模或找规律。

不要什么时候用：没有几何或浮点要求时不要翻；复杂计算几何不是本资料主攻方向，优先拿基础分。

复杂度：基础公式多为 $O(1)$ ；枚举点对/线段常见 $O(n^2)$ ；二分答案多乘 $\log$ 精度。

依赖的标准容器：点集可用 vector<Point>；浮点使用 double，整数叉积使用 long long 或 __int128。

接口：点结构、叉积、点积、距离平方、两倍面积、EPS 比较、取整公式和建模路由。

1. 基础计算几何公式

题面触发词：

- 点、线段、直线、多边形。
- 距离、面积、方向、叉积。
- 判断点在线段上。
- 多边形面积、三角形面积。

使用条件：

- 坐标是整数时，方向判断优先用整数叉积。
- 面积可能是 .5，可用两倍面积避免浮点。
- 涉及圆、角度、距离时通常用 double。

公式/结论：

向量 $AB = B - A$

点积 $\text{dot}(a, b) = a_x \cdot b_x + a_y \cdot b_y$

叉积 $\text{cross}(a, b) = a_x \cdot b_y - a_y \cdot b_x$

$\text{cross}(B-A, C-A) > 0$: C 在 AB 左侧

$\text{cross}(B-A, C-A) < 0$: C 在 AB 右侧

$\text{cross}(B-A, C-A) = 0$: A, B, C 共线

三角形两倍有向面积：

$\text{cross}(B-A, C-A)$

多边形两倍有向面积：

$\sum \text{cross}(P[i], P[i+1])$

C++17 模板或计算方式：

```
using ll = long long;
```

```
struct Point {
    ll x, y;
};
```

```
Point operator-(Point a, Point b) {
    return {a.x - b.x, a.y - b.y};
}
```

```
ll dot(Point a, Point b) {
    return a.x * b.x + a.y * b.y;
}
```

```
ll cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}
```



```

}

ll cross(Point a, Point b, Point c) {
    return cross(b - a, c - a);
}

bool on_segment(Point a, Point b, Point p) {
    return cross(a, b, p) == 0 &&
        min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
        min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

__int128 polygon_area2(const vector<Point> &p) {
    int n = (int)p.size();
    __int128 s = 0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        s += cross(p[i], p[j]);
    }
    return s >= 0 ? s : -s;
}

```

常见坑:

- 叉积正负和点的顺序有关。
- 多边形面积公式要首尾相接。
- 坐标可达 $1e9$ 时叉积可达 $1e18$, 用 long long; 更大用 __int128。
- 判断线段相交还要处理共线和端点。

暴力/部分替代:

- 小坐标格点题可枚举网格点。
- 面积不会推时, 把多边形拆成三角形。
- 只判断矩形/水平垂直线段时, 写特判。

最小验错:

$A=(0,0)$, $B=(2,0)$, $C=(0,2)$

$\text{cross}(A,B,C)=4$, 三角形面积 =2

正方形 $(0,0)(1,0)(1,1)(0,1)$ 的 $\text{area2}=2$, 面积 =1

2. 浮点误差与取整

题面触发词:

- 实数、误差不超过 $1e-6$ 。
- 开方、三角函数、圆。
- 向上取整、向下取整。
- 二分答案输出小数。

使用条件:

- 浮点比较用 eps。
- 能用整数就不用浮点。
- 除法取整要分清正负和方向。

公式/结论:

$\text{abs}(a-b) \leq \text{eps}$ 认为相等。

$a < b - \text{eps}$ 认为 a 明显小于 b 。

正整数 b 下:

$\text{floor}(a/b)$ 对非负 a 是 a/b 。

$\text{ceil}(a/b)$ 对非负 a 是 $(a+b-1)/b$ 。

C++17 模板或计算方式:

```
const double EPS = 1e-9;
```

```
int sgn(double x) {
```

```

    if (x > EPS) return 1;
    if (x < -EPS) return -1;
    return 0;
}

long long floor_div(long long a, long long b) {
    assert(b != 0);
    __int128 aa = a, bb = b;
    __int128 q = aa / bb;
    __int128 r = aa % bb;
    if (r != 0 && ((r > 0) != (bb > 0))) q--;
    assert((__int128)LLONG_MIN <= q && q <= (__int128)LLONG_MAX);
    return (long long)q;
}

long long ceil_div(long long a, long long b) {
    assert(b != 0);
    __int128 aa = a, bb = b;
    __int128 q = aa / bb;
    __int128 r = aa % bb;
    if (r != 0 && ((r > 0) == (bb > 0))) q++;
    assert((__int128)LLONG_MIN <= q && q <= (__int128)LLONG_MAX);
    return (long long)q;
}

```

常见坑:

- 不要用 `double == double` 判断几何相等。
- `sqrt` 后再转整数可能因为误差少 1 或多 1, 要回查修正。
- `ceil((double)a/b)` 对大整数可能精度丢失。
- C++ 整数除法对负数是向 0 截断, 不是向下取整。

暴力/部分分替代:

- 小范围答案可以整数枚举, 不用浮点二分。
- 几何距离小数据可暴力比较平方距离, 避免开方。

最小验错:

```

sgn(1e-10)=0
ceil_div(5,2)=3
floor_div(-3,2)=-2
ceil_div(-3,2)=-1

```

3. 常见数学建模路由

题面触发词:

- 看起来不是裸算法, 而是“规律、公式、证明、构造”。
- 数据范围很大, 普通模拟/DP 不可能。
- 出现周期、余数、整除、组合、期望、必胜。
- 让求第 n 项、方案数、最少/最多次。

使用条件:

- 先根据数据范围判断是否必须找数学规律。
- 把题面关键词映射到本卷模块。
- 如果正解推不出, 先写暴力/部分分并保留输入整理。

公式/结论:

路由口诀:

```

余数/周期 -> 同余、gcd/lcm、CRT
除法取模 -> 先问 mod 是否质数, 再选逆元
质数/因子 -> 筛法、分解、phi/mu
选法/路径 -> 组合数、二项式、Catalan、容斥
至少/没有/并集 -> 容斥
必然存在 -> 抽屉
第 n 项且 n 巨大 -> 递推、矩阵快速幂

```

随机平均 -> 概率期望
两人轮流 -> 博弈论
点线面 -> 叉积、点积、面积
小数答案 -> EPS、二分、取整

C++17 模板或计算方式:

建模时先写四行:

1. 变量是什么?
2. 目标是什么?
3. 限制是什么?
4. 数据范围允许什么复杂度?

再决定:

能模拟吗?

能 DP 吗?

是否有周期/组合/同余/递推公式?

常见坑:

- 没看数据范围, 写了会超时的模拟。
- 看到“取模”就默认模数是质数。
- 把题面 1-index/0-index、闭区间/开区间混淆。
- 公式只在互质、质数、独立等条件下成立, 却忘记检查。

暴力/部分替代:

- 小数据枚举所有对象, 记录结果找规律。
- 写 `solve_bruteforce()` 保底, 正解推出来后替换主函数。
- 若只有大数据难, 做特殊情况: 全相等、全 0、树退化链、模数为质数等。

最小验错:

题面: $n \leq 1e18$, 求 Fibonacci 第 n 项 mod $1e9+7$

路由: 第 n 项 + n 巨大 + 固定线性递推 -> 矩阵快速幂

题面: 很多条件中至少满足一个

路由: 至少一个/并集 -> 容斥

题面: 两人轮流取石子, 不能取者输

路由: 博弈论 win/lose 或 SG

MATH-LA-00 基础线性代数

模块编号: MATH-LA-00

模块名称: Matrix、Gaussian Elimination 与 XOR Basis

标签: [数学][线性代数][矩阵][高斯消元][线性基][异或]

一句话用途: 处理矩阵乘法、线性方程组、模意义方程组和异或最大值等基础线性代数题。

题面触发词:

- 矩阵、矩阵乘法、单位矩阵。
- 解线性方程组。
- 高斯消元、唯一解、无解、无穷多解。
- 答案对质数取模。
- 异或最大值、选若干数异或、线性基。

什么时候用:

- 方程是一次线性的: $a_1 \cdot x_1 + \dots + a_m \cdot x_m = b$ 。
- 小维度矩阵需要乘法或快速幂前置。
- 异或题允许从一堆数中任选若干个。
- 模意义高斯消元的模数通常是质数。

不要什么时候用:

- 方程有乘积、平方、max/min 等非线性项。
- 浮点方程精度要求极高，普通 double 不够。
- 模数不是质数且需要除法，不能直接用费马逆元。
- 异或线性基只处理 xor，不处理普通加法最大子集和。

复杂度：

- 矩阵乘法： $O(n*m*p)$ 。
- 高斯消元： $O(n*m^2)$ ，方阵常写作 $O(n^3)$ 。
- 异或线性基插入/查询： $O(\text{LOG})$ 。

数据范围参考：

- 高斯消元 $n \leq 200$ 比较常见。
- 矩阵快速幂维度通常 ≤ 50 。
- long long 异或线性基用 $\text{LOG} = 63$ ，覆盖非负 signed long long 的 bit 0..62。

依赖的标准容器：

- 1-index 矩阵：vector<vector<ll>> a(n + 1, vector<ll>(m + 1))。
- 增广矩阵：a[i][1..m] 是系数，a[i][m+1] 是常数。

输入如何整理：

```
int n, m; // n 个方程, m 个未知数
cin >> n >> m;
vector<vector<double>> a(n + 1, vector<double>(m + 2));
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m + 1; j++) cin >> a[i][j];
}
```

接口：

```
Matrix mul(A,B,mod)
gauss_double(a,x) -> 0 无解, 1 唯一解, 2 无穷多解
gauss_mod(a,x,MOD) -> 0 无解, 1 唯一解, 2 无穷多解
XorBasis.insert(x), max_xor(seed), can(x)
```

输出能力：

- 矩阵乘积。
- 实数线性方程组解。
- 质数模意义下线性方程组解。
- 可选子集异或出的最大值、某个值是否能表示。

下游可接：

- 矩阵快速幂。
- 概率期望方程。
- 图论 Kirchhoff 矩阵树，低优先级。
- 异或贪心、异或路径。

可拼接模块：

- MATH-LA-00 + MATH-02 FastPower。
- MATH-LA-00 + MATH-05 MatrixFastPower。
- MATH-LA-00 + GRAPH 异或环线性基。

矩阵基础模板：

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct Matrix {
    int n, m;
    vector<vector<ll>> a;

    Matrix(int n_ = 0, int m_ = 0) {
        n = n_;
        m = m_;
        a.assign(n + 1, vector<ll>(m + 1, 0));
    }
};
```

```

    }

    static Matrix identity(int n) {
        Matrix I(n, n);
        for (int i = 1; i <= n; i++) I.a[i][i] = 1;
        return I;
    }
};

Matrix mul(const Matrix &A, const Matrix &B, ll mod) {
    assert(A.m == B.n && mod > 0);
    Matrix C(A.n, B.m);
    for (int i = 1; i <= A.n; i++) {
        for (int k = 1; k <= A.m; k++) {
            if (A.a[i][k] == 0) continue;
            for (int j = 1; j <= B.m; j++) {
                C.a[i][j] = (C.a[i][j] + (__int128)A.a[i][k] * B.a[k][j]) % mod;
                if (C.a[i][j] < 0) C.a[i][j] += mod;
            }
        }
    }
    return C;
}

```

高斯消元模板: double

```

#include <bits/stdc++.h>
using namespace std;

const double EPS = 1e-9;

// a 是 1-index 增广矩阵: n 行, m 个未知数, 常数列在 m+1.
// 返回 0 无解, 1 唯一解, 2 无穷多解.
int gauss_double(vector<vector<double>> a, vector<double> &x) {
    int n = (int)a.size() - 1;
    int m = (int)a[1].size() - 2;
    vector<int> where(m + 1, -1);

    int row = 1;
    for (int col = 1; col <= m && row <= n; col++) {
        int sel = row;
        for (int i = row; i <= n; i++) {
            if (fabs(a[i][col]) > fabs(a[sel][col])) sel = i;
        }
        if (fabs(a[sel][col]) < EPS) continue;

        swap(a[sel], a[row]);
        where[col] = row;

        double div = a[row][col];
        for (int j = col; j <= m + 1; j++) a[row][j] /= div;

        for (int i = 1; i <= n; i++) {
            if (i == row) continue;
            if (fabs(a[i][col]) < EPS) continue;
            double factor = a[i][col];
            for (int j = col; j <= m + 1; j++) {
                a[i][j] -= factor * a[row][j];
            }
        }
        row++;
    }

    for (int i = 1; i <= n; i++) {

```

```

    bool all_zero = true;
    for (int j = 1; j <= m; j++) {
        if (fabs(a[i][j]) > EPS) all_zero = false;
    }
    if (all_zero && fabs(a[i][m + 1]) > EPS) return 0;
}

x.assign(m + 1, 0);
for (int col = 1; col <= m; col++) {
    if (where[col] != -1) x[col] = a[where[col]][m + 1];
}

for (int col = 1; col <= m; col++) {
    if (where[col] == -1) return 2;
}
return 1;
}
}

```

模意义高斯消元骨架：质数模

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll norm(ll x, ll mod) {
    x %= mod;
    if (x < 0) x += mod;
    return x;
}

ll mod_pow(ll a, ll e, ll mod) {
    ll res = 1 % mod;
    a = norm(a, mod);
    while (e > 0) {
        if (e & 1) res = (ll)((__int128)res * a % mod);
        a = (ll)((__int128)a * a % mod);
        e >>= 1;
    }
    return res;
}

// MOD 必须是质数; a 是 1-index 增广矩阵。
// 返回 0 无解, 1 唯一解, 2 无穷多解。
int gauss_mod(vector<vector<ll>> a, vector<ll> &x, ll MOD) {
    int n = (int)a.size() - 1;
    int m = (int)a[1].size() - 2;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m + 1; j++) a[i][j] = norm(a[i][j], MOD);
    }

    vector<int> where(m + 1, -1);
    int row = 1;

    for (int col = 1; col <= m && row <= n; col++) {
        int sel = 0;
        for (int i = row; i <= n; i++) {
            if (a[i][col] != 0) {
                sel = i;
                break;
            }
        }
        if (sel == 0) continue;
        swap(a[sel], a[row]);

        swap(a[sel], a[row]);
    }
}

```

```

where[col] = row;

ll inv = mod_pow(a[row][col], MOD - 2, MOD);
for (int j = col; j <= m + 1; j++) {
    a[row][j] = (ll)((__int128)a[row][j] * inv % MOD);
}

for (int i = 1; i <= n; i++) {
    if (i == row || a[i][col] == 0) continue;
    ll factor = a[i][col];
    for (int j = col; j <= m + 1; j++) {
        ll sub = (ll)((__int128)factor * a[row][j] % MOD);
        a[i][j] = norm(a[i][j] - sub, MOD);
    }
}
row++;
}

for (int i = 1; i <= n; i++) {
    bool all_zero = true;
    for (int j = 1; j <= m; j++) {
        if (a[i][j] != 0) all_zero = false;
    }
    if (all_zero && a[i][m + 1] != 0) return 0;
}

x.assign(m + 1, 0);
for (int col = 1; col <= m; col++) {
    if (where[col] != -1) x[col] = a[where[col]][m + 1];
}

for (int col = 1; col <= m; col++) {
    if (where[col] == -1) return 2;
}
return 1;
}

```

异或线性基模板:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct XorBasis {
    static const int LOG = 63;
    ll b[LOG];

    XorBasis() {
        memset(b, 0, sizeof(b));
    }

    bool insert(ll x) {
        for (int i = LOG - 1; i >= 0; i--) {
            if (((x >> i) & 1LL) == 0) continue;
            if (b[i] == 0) {
                b[i] = x;
                return true;
            }
            x ^= b[i];
        }
        return false;
    }
}

ll max_xor(ll seed = 0) const {

```

```

    ll res = seed;
    for (int i = LOG - 1; i >= 0; i--) {
        res = max(res, res ^ b[i]);
    }
    return res;
}

bool can(ll x) const {
    for (int i = LOG - 1; i >= 0; i--) {
        if ((x >> i) & 1LL) == 0) continue;
        if (b[i] == 0) return false;
        x ^= b[i];
    }
    return true;
}

int rank() const {
    int res = 0;
    for (int i = 0; i < LOG; i++) {
        if (b[i]) res++;
    }
    return res;
}
};

```

调用示例:

```

vector<double> x;
int type = gauss_double(a, x);
if (type == 1) {
    for (int i = 1; i < (int)x.size(); i++) cout << x[i] << '\n';
}

```

```

XorBasis xb;
for (int i = 1; i <= n; i++) xb.insert(a[i]);
cout << xb.max_xor() << '\n';

```

常见坑:

- 高斯消元读的是增广矩阵, 列数要开到 $m+2$ 。
- double 比较必须用 EPS, 不要直接 $== 0$ 。
- 模意义高斯消元只有在 pivot 可逆时才能除; 质数模下非零元素都可逆。
- 模数不是质数时, $\text{pow}(a, \text{MOD}-2)$ 求逆元是错的。
- 异或线性基默认处理非负 long long; 如果题目有符号数, 先确认位数和比较规则。
- 线性基插入失败表示这个数可由已有数异或出来, 不代表它没用错了。

暴力/部分分替代:

- 方程数量很小: 枚举未知数或手推。
- 模方程未知数很少且取值范围小: DFS 枚举。
- 异或最大值 $n \leq 24$: 枚举所有子集。
- 只需要矩阵乘法, 不需要快速幂: 直接三重循环。

升级方向:

- 矩阵基础 $\rightarrow$ 矩阵快速幂。
- 高斯消元 $\rightarrow$ 期望 DP 方程组。
- 异或线性基 $\rightarrow$ 图上异或环、路径最大异或。
- 模高斯 $\rightarrow$ 矩阵求逆、行列式, 低优先级。

最小测试样例:

double 方程:

$x + y = 3$

$x - y = 1$

解: $x=2, y=1$

线性基:

1 2 3

最大异或: 3